

RAGLoom

A visual RAG pipeline editor for local, on-device product Q&A.

Drop in product spec documents, ask questions in natural language, and get grounded answers — generated entirely on-device, with no cloud dependency. Built in Python, FastAPI, React and ChromaDB on Ollama. No LangChain.

THE PROBLEM

How do you run a trustworthy product-Q&A demo on a laptop, with no internet and a small model — without it inventing specs or recommending a competitor?

Trade-show booths and retail kiosks want a live, conversational product assistant — and for a PC OEM, running the model *on the device itself* doubles as proof of AI-PC performance. But the local-only constraint forces a small model (gemma3:4b), and small models hallucinate, misread numbers, and can't be trusted to "just follow instructions" on the things that matter at a booth: never quoting a price, never naming a competitor, never recommending a 1.8 kg laptop for a "under 1 kg" request.

THE APPROACH

RAGLoom moves every high-stakes decision out of the prompt and **into code**. Four deterministic gates short-circuit a query *before* it can reach the generator, each firing on a different signal and composing without overlap. What's left for the LLM is the one thing it's good at: phrasing a grounded answer from retrieved context.

The whole pipeline is a **visual, drag-and-drop node graph** — so a PM can open the editor mid-demo, point at the safety layer, and watch a blocked query light up amber across the canvas in real time.

THE SAFETY LAYER — POLICY IN CODE, NOT IN THE PROMPT

Guardrail

Keyword block list (e.g. competitor brands) with word-boundary matching → canned refusal.

PriceGuard

Regex price-intent detector (price / MSRP / 多少錢...) → bilingual refusal, pre-retrieval.

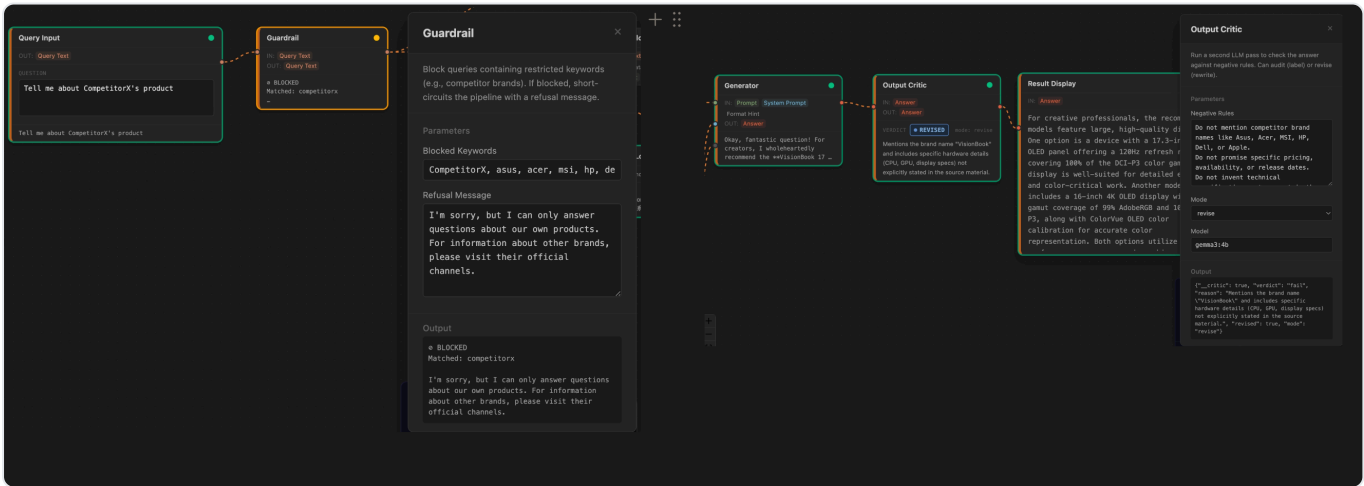
ScopeGate

Semantic on/off-topic check via anchor embeddings. Survives bridge attacks pure cosine can't.

ConstraintFilter

Numeric spec gate ("under 1kg"). Resolves each candidate's canonical spec, drops violators.

The common thread: at small-model scale, "trust the model to follow instructions" is unreliable for refusals and numeric comparison. A 4B model will mark *1.8 kg < 1 kg* as passing — so the comparison never reaches it. Each gate degrades safely and, when it fires, returns a canned bilingual refusal rather than handing the model an empty context to hallucinate over.



Builder's view — a blocked query short-circuits the downstream nodes across the pipeline, visible mid-demo. The safety layer is demonstrable, not buried in a config file.

THE PRODUCT

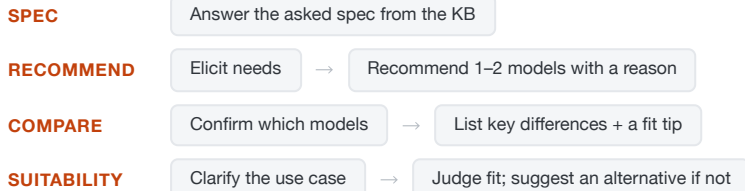
WHO IT'S FOR

End users / booth visitors — a Chat UI: ask a question, get a formatted, grounded answer with a retrieval-details panel and (in chatbot mode) an animated avatar that reflects the model's self-reported emotion. Replies always come back in the visitor's language.

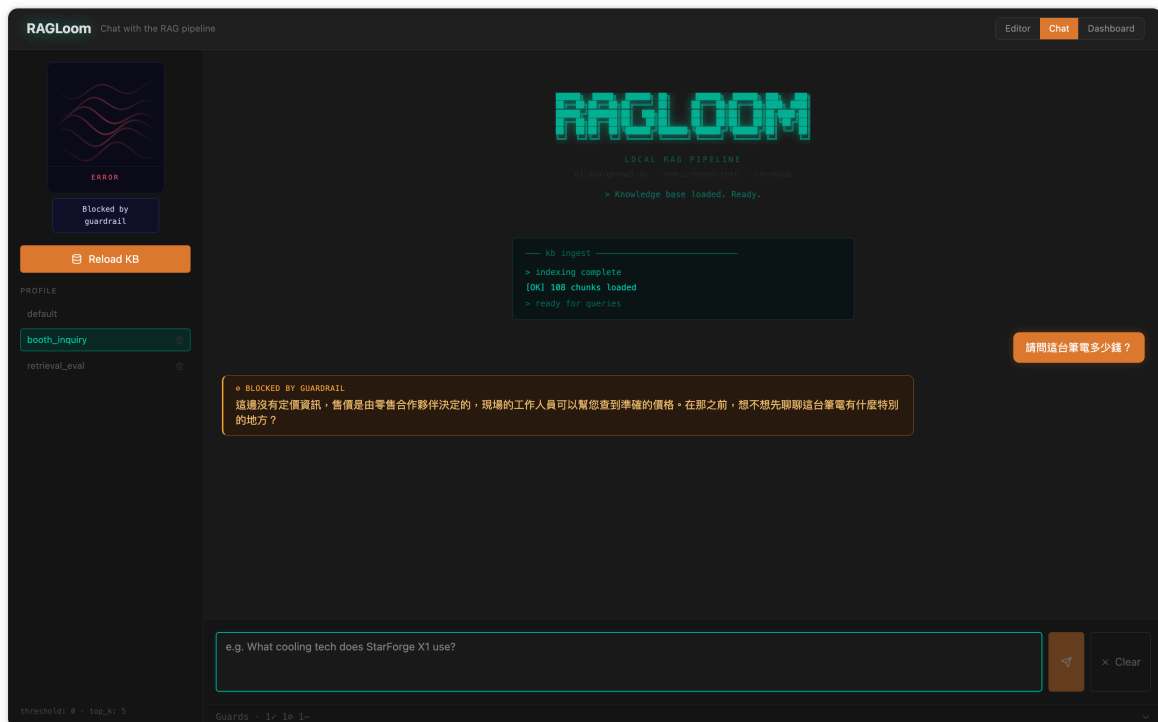
Builders & operators — a Node Editor to assemble, rewire and A/B the pipeline, plus an in-editor batch eval harness to measure retrieval quality before shipping a graph variant.

CONVERSATIONAL INQUIRY FLOW — A BOOTH ASSISTANT, NOT A SEARCH BOX

IntentRouter classifies each visitor turn (one small LLM call) into one of four inquiry intents, re-detected *every turn* so a topic-hopping visitor is followed immediately. **DialogueFlow** then runs the matching multi-stage script, advancing the conversation toward a recommendation:



Guard-aware state. Stage and intent only advance on a *committed* turn — one where the generator actually produced an answer. A guard short-circuit freezes the whole funnel, so refusals and off-topic turns never pollute the conversation's stage or history. The single advance gate (*decide_advance*) is the one place designed to swap LLM judgement for deterministic slot logic — hardening a script without touching anything else.



Visitor's view — a price query is caught by the safety layer and answered with a canned bilingual refusal, never a hallucinated number. The same block that lit up the node graph, seen from the end-user side.

CAPABILITIES

WHAT'S IN THE BOX

● **Visual node editor**

27 node types across ingest / query / eval. Real-time per-node execution status over WebSocket; first-class edge editing with one-value-per-input semantics that mirror the engine.

● **Dialogue-aware routing**

IntentRouter + DialogueFlow turn the pipeline into a guided booth conversation — four inquiry scripts with guard-frozen state, shipped as the active *booth_inquiry* profile.

● **Metadata-filtered retrieval**

Docs tagged with a product_id at ingest. A ProductSelector routes point-queries to a single product (rule mode = zero LLM latency); comparisons fall through to broad search.

● **Retrieval Judge**

An LLM rerank pass drops retrieved chunks that don't actually answer the query — catching polarity / negation misses pure cosine retrieval can't. Degrades to keep-all on error.

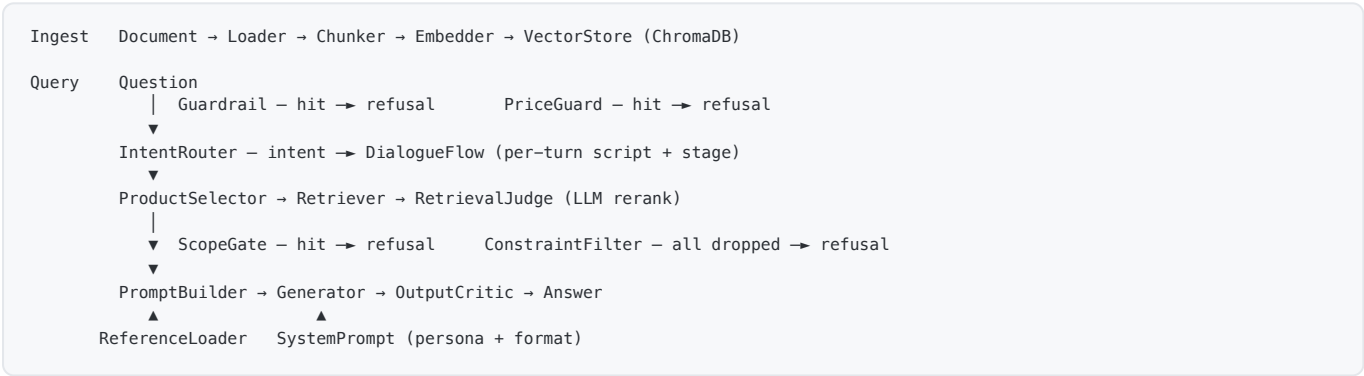
● **Always-on reference data**

A static comparison table injected into every prompt, guaranteeing broad coverage for comparison queries independent of vector retrieval.

● **Persona presets & structured output**

professional / chatbot registers tuned to a trade-show promoter voice. Chatbot mode emits grammar-constrained JSON {reply, emotion}, smart-rendered as a themeable avatar.

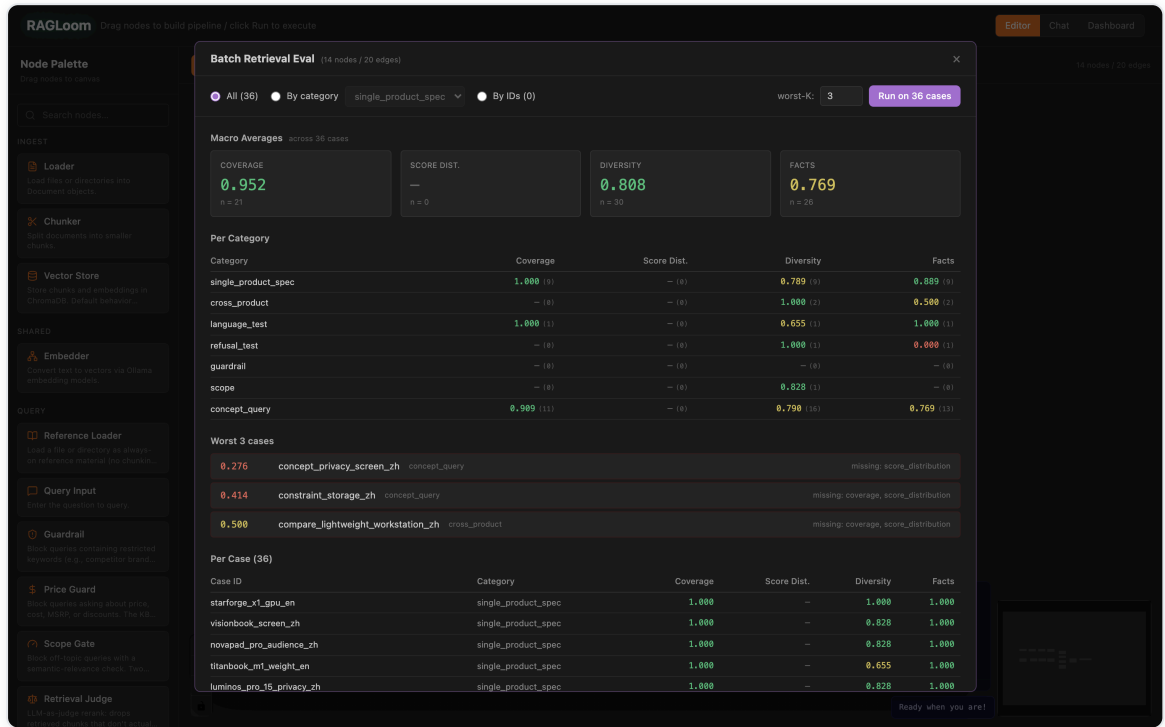
ARCHITECTURE



QUALITY & UNDER THE HOOD

QUALITY & EVALUATION

- **Golden-set eval with LLM-as-judge** — a curated regression set, audited by a second LLM that returns explicit *hallucinated_claims* lists. Gates on the binary signal (claims empty / non-empty), not noisy float scores, so same-commit reruns don't flicker pass/fail.
- **In-editor batch eval** — a dedicated eval-node family (Hit@K coverage, score distribution, diversity, facts coverage) plus a Run Batch sweep over golden-set cases, with macro averages, per-category breakdown and worst-K ranking — fast, deterministic, retrieval-only.
- **Hardened local API** — every endpoint requires an auto-generated local token, CORS is locked to the local origin, file-path params are confined to an allowlist, and batch eval is bounded. A stray malicious browser tab can't drive your local pipeline.



In-editor batch eval — macro averages, per-category breakdown, worst-K and a full per-case table swept across the golden set, without touching the chat path.

TECH STACK

Python · FastAPI

React · Vite · TypeScript

ChromaDB

Ollama (gemma3:4b)

nomic-embed-text

WebSocket

No LangChain

AT A GLANCE

27
NODE TYPES

4
CODE-LEVEL GATES

0
CLOUD CALLS